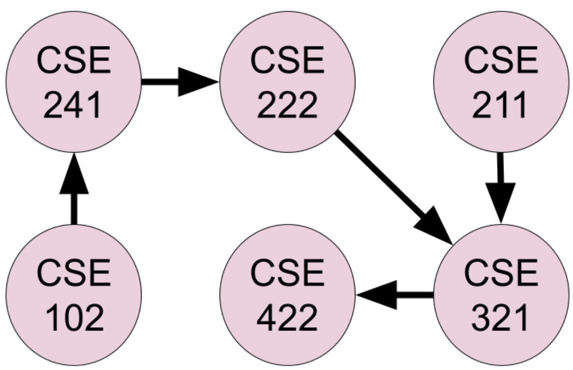
CSE 321 Study Question Examples

1. A directed acyclic graph (DAG) is a directed graph with no directed cycles. A DAG has at least one node with indegree (number of incoming edges) 0 and one node with outdegree (number of outgoing edges) 0.

   Topological sorting of a DAG is a linear ordering of nodes such that for every edge ab (an edge from node a to node b), a comes before b in the ordering. A DAG may have more than one topological order.

   Example: The following graph presents some of the courses in our department. The directed edges indicate the prerequisites. For instance, a student cannot enroll in $CSE321$ without successfully passing the courses $CSE211$ and $CSE222$. On the other hand, $CSE102$ and $CSE211$ do not have any prerequisites. Topological sortings of this graph should demonstrate the order of these courses such that if a student follows that ordering, they meet all of the prerequisite requirements.

   Such orderings are:

   - $CSE102 \rightarrow CSE241 \rightarrow CSE222 \rightarrow CSE211 \rightarrow CSE321 \rightarrow CSE422$
   - $CSE102 \rightarrow CSE241 \rightarrow CSE211 \rightarrow CSE222 \rightarrow CSE321 \rightarrow CSE422$
   - $CSE102 \rightarrow CSE211 \rightarrow CSE241 \rightarrow CSE222 \rightarrow CSE321 \rightarrow CSE422$
   - $CSE211 \rightarrow CSE102 \rightarrow CSE241 \rightarrow CSE222 \rightarrow CSE321 \rightarrow CSE422$

   (a) Construct a DFS-based algorithm to obtain such an ordering for a given DAG.

   (b) Construct a non-DFS-based algorithm to obtain such an ordering for a given DAG.

   PS: The algorithms should find a single solution. You are not asked to find all possible orderings.

2. Design an algorithm to calculate a^n, where $a \in Z$ and $n \in Z^+$, with a worst-case time complexity of $O(logn)$.

3. Design an algorithm to solve 9x9 sudoku game by using exhaustive search.

4. Design a decrease-and-conquer algorithm for generating all combinations of k items chosen from n, i.e., all k-element subsets of a given n element set. Iis your algorithm a minimal-change algorithm?

5. (a) Give an explanation of the relation between brute force and exhaustive search.

(b) Learn about the basic idea of Caesar's Cipher and AES. Are they vulnerable to brute force attacks? Explain your answer.

(c) Why does the naive solution to primality testing (for input n, checking if $x \in \{2, 3, \ldots, n-1\}$ divides n) grow exponentially?